

Interconnected AI Team: Concept

Goal

Build a small team of AI agents for Interconnected that runs on Claude Code. Each agent has one clear job. They share knowledge through a single repo instead of long-running chat threads. Start small and add roles only once each one proves itself.

This document describes the concept only. Which tasks and roles come first is a separate decision.

Core idea: the repo is the company brain

- A private GitHub repo is the single source of truth: company context, instructions, history, and direction.
- Every agent session reads the repo before working and writes back what it did or learned.
- **Threads are disposable; files are the job.** No knowledge should live only in a chat. If it isn't in the repo, the team doesn't know it.

What the repo holds (conceptually)

- **Company context:** what Interconnected is, who we sell to, messaging, objections, pricing, and the rules every session follows.
- **Direction:** current priorities, decisions made, what's working, and what to stop doing. Every role reads this first, and it is the main lever for steering the team.
- **Role definitions:** one instruction file per role, covering its job, how it does it, what tools it may use, and what it must never do.
- **Reusable procedures:** step-by-step skills that any role can use.
- **Work records:** outputs (e.g., research, drafts) and statuses showing what's been done.
- **Logs:** a short entry from every run covering what was done and what changed.

How work runs

- **Recurring tasks** run on a schedule as Claude Code routines. Each run starts fresh, reads the repo, does its job, and writes results back.
- **One-off tasks** start in a new session pointed at the repo.
- **Changing how a role works** means editing its instruction file, not returning to an old thread. The next run picks up the change automatically.
- **Parallel work** (e.g., researching many items at once) uses subagents inside a session. Each gets its own context and returns a summary.
- **End every meaningful session** with "update the repo with anything we decided."

How agents coordinate

- Only through the repo and simple status fields (e.g., `needs-work` → `draft-ready` → `approved` → `done`).
- Agents do not talk to each other directly. One role's output becomes the next role's input through files and statuses.

Guardrails

- **A human approves anything that leaves the building.** Agents draft; Noah sends, publishes, or spends.
- No agent gets payment methods or unnecessary access to sensitive data.
- An agent that reads untrusted content (websites, inbound email) must not also be able to send messages externally, to guard against prompt injection.
- Credentials live in secrets, never in instruction files or prompts.
- Every run leaves a log, and a claim of completed work must point to its output (file, link, or diff).

Growth rules (guard against over-building)

- Start with one or two roles at most.
- Add a new role only after the current ones have run for about two weeks with few outputs needing major rewrites.
- Don't add inter-agent messaging, orchestration dashboards, or "manager" agents until there is a concrete coordination problem they would solve.

- Start on the existing Claude plan. Move a role to API billing with spend caps once it runs daily in production.

Later, not now

- More roles (content, support, etc.)
- A chat front end (e.g., Slack) over the same repo and roles
- Adapting the same approach for PAX, with stricter data access and approval controls